Session 2 Mean median and mode

Practical Exercises in Descriptive Statistics & Data Analysis using Python

Problem 1: Spotify Playlist Daily Stats (`playlist_stats.py`)

Task: Create a Python script that takes daily Spotify play counts and computes the mean daily plays.

```
# playlist_stats.py import statistics def calculate_playlist_stats(): daily_plays = [120, 135, 150, 200, 120, 90, 200] # Calculate arithmetic mean mean_plays = statistics.mean(daily_plays) print(f"Daily Play Counts: {daily_plays}") print(f"Mean (Average) Daily Plays: {mean_plays:.2f}") if __name__ == "__main__": calculate_playlist_stats()
```

PROGRAM OUTPUT

Daily Play Counts: [120, 135, 150, 200, 120, 90, 200]
Mean (Average) Daily Plays: 145.00

Problem 2: Zomato Order Delivery Times (Median)

Task: Given delivery times in minutes `[30, 25, 40, 35, 30, 45, 30]`, compute and display the median delivery time.

```
import statistics delivery_times = [30, 25, 40, 35, 30, 45, 30] median_time = statistics.median(delivery_times) print(f"Sorted Order Times: {sorted(delivery_times)}") print(f"Median Delivery Time: {median_time} minutes")
```

PROGRAM OUTPUT

Sorted Order Times: [25, 30, 30, **30**, 35, 40, 45]
Median Delivery Time: 30 minutes

Problem 3: Flipkart Product Ratings (Mode Function)

Task: Implement a reusable function `most_common_rating(ratings)` that calculates the mode rating.

```
import statistics def most_common_rating(ratings): if not ratings: return None return statistics.mode(ratings) flipkart_ratings = [5, 4, 4, 3, 5, 4, 2, 4] mode_rating = most_common_rating(flipkart_ratings) print(f"Flipkart Ratings: {flipkart_ratings}") print(f"Most Common Rating (Mode): {mode_rating}")
```

Problem 4: YouTube Views & Outlier Impact Analysis

Task: Compare central tendencies across three YouTube channels to analyze how extreme viral video view counts impact the mean versus the median.

```
import statistics channel_a = [1050, 1100, 1020, 1080, 1060] # Steady, standard growth channel_b = [100, 120, 110, 105, 50000] # Massive viral outlier (50k) channel_c = [500, 500, 500, 600, 550] # Consistent viewership def analyze_channel(name, views): mean_val = statistics.mean(views) med_val = statistics.median(views) mode_val = statistics.mode(views) return (mean_val, med_val, mode_val)
```

Channel	Raw View Data	Mean	Median	Mode	Outlier Sensitivity
Channel A	[1050, 1100, 1020, 1080, 1060]	1,062.0	1,060.0	1,050	**Low Variance**
Channel B	[100, 120, 110, 105, 50000]	**10,087.0**	**110.0**	100	**Severely Distorted**
Channel C	[500, 500, 500, 600, 550]	530.0	500.0	500	**Stable**

STATISTICAL CONCLUSION

Channel B is overwhelmingly affected by the outlier. While 80% of its videos hover around 100–120 views, the single viral hit (50,000 views) pulls the mean up to 10,087. The **median (110)** provides a much more accurate representation of typical performance.

Problem 5: Real Dataset Analysis with Pandas (Titanic Dataset)

Task: Load a real-world dataset, select a numeric feature column (`Fare`), and calculate descriptive statistics using `pandas`.

```
import pandas as pd # Load public Titanic Passenger dataset from repository url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv" df = pd.read_csv(url) # Compute metrics for numeric column 'Fare' mean_fare = df['Fare'].mean() median_fare = df['Fare'].median() mode_fare = df['Fare'].mode()[0] print(f"Titanic Fare Analysis:\nMean: ${mean_fare:.2f} | Median: ${median_fare:.2f} | Mode: ${mode_fare:.2f}")
```

MEAN FARE
$32.20
Pulled upward by 1st class luxury tickets

MEDIAN FARE
$14.45
True midpoint of overall ticket prices

MODE FARE
$8.05
Most frequent standard 3rd class ticket